

Session Walkthrough: Induced-Taxonomy LLM vs. KMeans

Canadian Hansard, 1920–1949

Mathias Bühler

2026-05-26

The big picture

Last session (2026-05-19) we built a one-shot LLM classifier: we wrote down ten categories by hand, passed them to Gemini as an `Enum`, and asked the model to pick one per speech. That worked, but it had two problems. First, the categories were ours, not the data's: *Wartime*, *Infrastructure*, *Health care*, *Education*, *Income and Tax*, *International relations*, *Budget*, *Trade*, *Immigration*, *Other*. The *Other* bucket absorbed 30–40% of speeches because our schema did not reflect what Parliament actually spent its time on. Second, we had no baseline to compare against. We trusted the LLM's labels without checking whether a vocabulary-only method would tell the same story.

Today's session built two new pipelines in parallel and a paper that compares them. The arc:

1. **Induce** a ten-category taxonomy from the data instead of writing it down. First ask the LLM for a free-text noun phrase per speech (162k phrases, 130k unique), then ask a stronger LLM to read the 2,000 most frequent phrases and propose ten categories with definitions, then ask the cheap LLM to assign each unique phrase to one category.
2. **Cluster** the same speeches with TF-IDF + MiniBatchKMeans ($k=20$) as an unsupervised baseline.
3. **Compare** the two pipelines on the WWII window, the Bank of Canada Act (1933–35), the Wheat Board (1935), and federal-provincial relations (1930–35). Write up the comparison as a 10-page seminar paper (`draft/2605/report.tex`).

The headline finding is that the two pipelines agree on the *timing* and *direction* of every historical episode in the period but disagree on *magnitudes* in a way that follows mechanically from how each represents meaning. The WWII spike is the clean test: the LLM's *Veterans' and Military Affairs* category jumps from 2.8% in 1935 to 34% in 1942; the three closest KMeans clusters combined peak around 12%. Neither answer is wrong. They measure different things.

1 Step 1 — Free-text topic extraction (pass 1)

Script: `src/classify_speeches_gemini_2605.py`

The structural change from last session is that we drop the `response_schema=Category` constraint. Instead of forcing the model to pick from ten fixed labels, we ask for a short noun phrase. The system prompt is:

```
You are a research assistant analysing speeches from the Canadian House
of Commons, 1920–1949. Read the speech and return a SHORT NOUN PHRASE
```

(maximum 8 words, no full sentence, no trailing period) describing the substantive topic the speaker addresses.

Rules:

- Be specific: "old-age pensions" not "social policy";
"St. Lawrence Seaway construction" not "infrastructure".
- Lower-case unless a proper noun requires capitalisation.
- Ignore procedural openings ("Mr. Speaker, ...") and classify by the substantive topic.
- If the speech spans several topics, pick the one with the most content.
- If you genuinely cannot identify a topic, return "unclear".

Return only the noun phrase, nothing else.

Why temperature 1.0. Last session we used temperature 0 because we wanted deterministic picks from a fixed list. Here we want the model to find specific phrasings, so we raise temperature to 1.0. The cost is that the same speech can in principle get different phrasings across runs; the benefit is that the model commits to a substantive topic instead of defaulting to a generic noun.

What we get. The output is `output/speeches.classified.2605.csv` with one row per speech: `basepk`, `topic`, `error`. Of 161,633 speeches, zero errors and zero empty topics in the production run. The vocabulary of topics is long-tailed: 129,899 distinct phrases, of which roughly 80% appear exactly once. The top 2,000 phrases cover only 12.4% of speeches. This is the central trade-off of free-text measurement: granular but not directly aggregable.

Operational notes that carry over from last session. Async with `asyncio.Semaphore(80)`, append-as-you-go to CSV with `FLUSH EVERY = 500`, resumable on `Ctrl-C` (re-reads the existing CSV, drops the `basepks` already done), exponential backoff with jitter on `429 / RESOURCE_EXHAUSTED`. Throughput is about 135 requests per second, full panel in roughly 20 minutes.

2 Step 2 — Induce the taxonomy from the data (pass 2, stage 1)

Script: `src/induce_taxonomy_gemini_2605.py`

This is the conceptual move of the session. We take the top 2,000 most frequent topic strings (with their frequency counts), pass them in a single call to `gemini-2.5-pro`, and ask the model to write down ten categories with one-sentence definitions. The output schema enforces exactly ten items:

```
RESPONSE_SCHEMA = {
    'type': 'object',
    'properties': {
        'categories': {
            'type': 'array',
            'minItems': 10,
            'maxItems': 10,
            'items': {
                'type': 'object',
                'properties': {
                    'name': {'type': 'string'},
```

```

        'definition': {'type': 'string'},
    },
    'required': ['name', 'definition'],
},
},
},
'required': ['categories'],
}

```

Why a stronger model here. The call is one-shot. The output drives every downstream label. We pay once for `gemini-2.5-pro` (a few cents) to get a taxonomy that fits the data we actually have, then use the cheap `flash-lite` model for the per-speech work. Temperature is 0; the call is deterministic conditional on the input prompt.

Why this is structurally better than last session’s hand-written list. The induced taxonomy is empirically grounded. The largest induced category turns out to be *Government and Parliamentary Affairs* (16.7% of speeches), which is what an empirically grounded residual looks like: it absorbs procedural and meta-parliamentary content that, in our last-session schema, was being split across *Other* and forced misclassifications. The categories the model proposed:

#	Category	Share
1	Government and Parliamentary Affairs	16.7%
2	Transportation and Communications	13.4%
3	Agriculture and Food Policy	11.9%
4	Veterans’ and Military Affairs	11.5%
5	Public Finance and Taxation	10.6%
6	Social Welfare and Labour	9.2%
7	Justice, Constitution, and Indigenous Affairs	9.1%
8	International Trade and Tariffs	9.0%
9	Natural Resources and Intergovernmental Relations	5.7%
10	Banking and Monetary Policy	3.1%

What the induced taxonomy misses. Three themes that we might want to track separately get absorbed into broader categories: education scatters across Social Welfare/Labour and Justice/Constitution (residential schools); health care folds into Social Welfare/Labour; immigration folds into Justice/Constitution alongside citizenship law. The fix is cheap: hand-edit `output/topic_taxonomy_2605.json` and re-run the 1.6-minute assignment pass. The point of separating measurement from taxonomy is that taxonomy revisions are cheap.

3 Step 3 — Assign each topic to a category (pass 2, stage 2)

Script: `src/assign_categories_gemini_2605.py`

For each of the 129,899 unique topic strings, ask the cheap LLM to pick one category. Two efficiency moves:

1. **Assign on unique topics, not on speeches.** The call count drops from 161k to 130k. Then the per-speech label is a left-join on the topic field. Two speeches with the same topic string always receive the same category by construction.

2. **Batch 50 topics per call.** The prompt is a JSON array of 50 topic strings; the response schema enforces a same-length array of category-name enums. This drops the call count by another factor of 50, to about 2,600 calls. Throughput is around 1,360 topics per second; total wall time is about 1.6 minutes.

```
response_schema = {
    'type': 'array',
    'items': {'type': 'string', 'enum': category_names},
}
config = types.GenerateContentConfig(
    system_instruction=system_instruction,
    temperature=0.0,
    response_mime_type='application/json',
    response_schema=response_schema,
)
```

Why this works. Last session's lesson was that constrained generation beats prompt engineering. Here we constrain twice: the *taxonomy induction* call is constrained to return exactly ten name-plus-definition objects, and the *topic-to-category* call is constrained to return a same-length array of category-name enums. The model is physically incapable of returning a category we did not define, and incapable of returning the wrong number of labels per batch. No regex, no fallback, no malformed-output handling.

Output. `output/topic-to-category_2605.csv` has 129,899 rows of `topic`, `category`. Joined back to `speeches_classified_2605.csv` on `topic`, we get the per-speech label that goes into the comparison.

4 Step 4 — KMeans baseline

Scripts: `src/cluster_speeches_2605.py`, `src/plot_clusters_2605.py`

The KMeans pipeline is the unsupervised baseline. The point of having it is not that we trust it more than the LLM; the point is that having a vocabulary-only baseline lets us isolate what the LLM contributes *on top of* word co-occurrence. The pipeline:

1. **Load and filter.** All 12 `input/Speeches*.xlsx` files, drop topic-header rows and missing speech text, deduplicate, drop speeches with fewer than 100 words. The 100-word floor removes 77% of the de-duplicated corpus and is by far the most consequential filter. Without it, the corpus is dominated by procedural interjections with no distinguishing vocabulary, and the algorithm collapses 57% of mass into one undifferentiated residual.
2. **Two-stage POS filter.** Stage A is a per-speech pass that drops short tokens, stopwords, a custom banlist, and tokens with no WordNet synset whose POS is noun or adjective. Stage B collects the surviving unstemmed tokens (about 41k unique terms) into a vocabulary and runs `nltk.pos_tag` once on the vocabulary — not per speech — keeping terms tagged `NN*` or `JJ*`. This costs one POS-tag call on the vocabulary instead of one call per speech.
3. **Stem and vectorize.** Porter-stem the surviving tokens, then TF-IDF with `ngram_range=(1,3)`, `min_df=10`, `max_df=0.5`. Matrix is 188,583 documents by 379,233 features (sparse).

4. **MiniBatchKMeans**, $k=20$. Full KMeans is wasteful at this scale (30+ minutes per iteration). MiniBatch finishes in seconds with comparable cluster purity.
5. **Plot**. Three figures: overall cluster sizes (`Cluster_Sizes_2605_KMEANS.png`), per-cluster share of yearly speeches faceted (`Cluster_Shares_by_Year_2605_KMEANS.png`), and a stacked-area plot of all clusters by year (`Cluster_Stacked_by_Year_2605_KMEANS.png`).

The banlist matters. The initial run with the user-supplied banlist collapsed 57% of mass into a residual cluster whose top terms were verbs (*inform*, *answer*, *agree*) that survived the WordNet check because they have noun senses. Adding the Perceptron POS pass on the vocabulary cut that to 45%. A further aggressive banlist (procedural noise like *hon*, *sir*, *madam*, *speaker*, *house*, *committee*, *order*; content-free generics like *time*, *year*, *day*, *matter*, *case*, *fact*, *point*; and high-frequency content words like *minister*, *people*, *work*, *government*, *member* that were drowning out everything else) brought the residual to 37%. This is a stylistic intervention: banning *minister*, *people*, *government* flattens part of the substantive signal. The trade-off is acceptable because the prime-minister cluster and the federal-provincial cluster then surface through other distinctive terms.

Tuning failures matter as much as the success. Three earlier runs preceded the final output. Run 1 ($k=10$, unigrams plus bigrams, no length filter): 57% residual. Run 2 (plus 100-word filter and Perceptron POS pass): 45% residual. Run 3 ($k=20$, bigrams plus trigrams only, aggressive banlist): 76% of mass in one cluster, eight others singletons. Removing unigrams in favor of bigrams plus trigrams seems intuitive (“we want compound topics like *wheat board*”) but the dense low-rank signal in unigrams is what keeps document vectors from collapsing onto each other in the sparser ngram space. Run 4 ($k=20$, unigrams plus bigrams plus trigrams, `min_df=10`, aggressive banlist, 100-word floor) is what we kept.

5 Step 5 — Compare

Output: `draft/2605/report.tex` and `draft/2605/report.pdf`

The comparison is structured around four historical episodes whose timing is uncontroversial and whose magnitude is the question:

1. **WWII (1939–1945).** LLM *Veterans’ and Military Affairs* rises from 2.8% in 1935 to 34% in 1942. The three closest KMeans clusters (wartime international, defence, wartime labour) summed reach about 12% at peak. The LLM magnitude is roughly three times the KMeans magnitude.
2. **Bank of Canada Act (1933–1935).** KMeans cluster 16 (*bank canada*, *credit*, *gold*, *central bank*) and LLM *Banking and Monetary Policy* (3.1% of corpus) show the same window with the same shape. The two pipelines agree.
3. **Wheat Board (1935).** KMeans splits the theme into two clusters (cluster 4: *canadian wheat board*, *bushel*, *grain*; cluster 8: *farmer*, *wheat*, *grain*, *western canada*) — a known KMeans pathology where the algorithm has no incentive to merge centroids that occupy distinct regions of the vocabulary space even when they cover the same theme. The LLM *Agriculture and Food Policy* category absorbs both.
4. **Federal-provincial relations (1930–1935).** KMeans cluster 18 and LLM *Natural Resources and Intergovernmental Relations* (5.7%) line up.

Why the WWII gap is mechanical, not a tuning failure. A speech that says “we must defend Canada” but uses budget vocabulary throughout will be assigned to *Veterans’ and Military Affairs* by the LLM and to the budget cluster by KMeans. The LLM reads the substance; KMeans reads the words. The two pipelines disagree most where the substantive topic crosses vocabulary boundaries (the war effort, which appears in budget, trade, and procedural speeches simultaneously) and agree where the substance is well-encoded by ordinary words (banking, the Wheat Board, federal-provincial relations).

The residual in two forms. KMeans cluster 2 (37% of mass) has top terms *canada*, *depart*, *act*, *countri* — the most generic content words that survive the banlist. There is no way to interrogate it short of a second clustering pass. The LLM *Government and Parliamentary Affairs* residual (17%) is different: its top topic strings are individually interpretable (electoral redistribution, civil-service salary deductions, House standing orders, continuation of wartime economic controls, National Emergency Transitional Powers Act). Roughly half is genuine procedural content; roughly half is topics that a slightly better taxonomy would route elsewhere.

6 How to replicate end-to-end

From a fresh clone, assuming `input/Speeches*.xlsx` and `input/Link.ID.csv` are in place:

```
# one-time, from prior sessions
python src/scrape_parlinfo.py # -> output/MPs.csv
python src/prepare_claude_v2.py # -> output/speeches_all.csv
python src/merge_speeches_mps.py # -> output/speeches_all_mps.csv

# today's work
# LLM pipeline (about 22 min end-to-end, a few dollars)
python src/classify_speeches_gemini_2605.py # -> speeches_classified_2605.csv
python src/induce_taxonomy_gemini_2605.py # -> topic_taxonomy_2605.json
python src/assign_categories_gemini_2605.py # -> topic_to_category_2605.csv

# KMeans baseline (about 3 min)
python src/cluster_speeches_2605.py # -> Speeches_clustered_2605.csv
python src/plot_clusters_2605.py # -> three PNGs

# Comparison paper
cd draft/2605 && pdflatex report.tex
```

7 The three ideas worth remembering

1. **Induce taxonomies from the data, do not write them down.** Last session’s hand-written ten-category list pushed 30–40% of speeches into *Other*; today’s data-induced ten-category list pushes only 17% into the largest residual, and that residual is itself made of interpretable topics rather than uncategorisable noise. The two-pass design (free-text noun phrase first, taxonomy and assignment second) also separates measurement from taxonomy, so taxonomy revisions cost two minutes of compute instead of a full re-classification.
2. **Constrained generation, again, at two layers.** The induction call is constrained to return exactly ten name-plus-definition objects. The assignment call is constrained to return a same-

length array of category-name enums. Both are checked by schema, not by regex on free text. Whenever the model has structured output to produce, force the structure with the schema, not with prompt politeness.

3. **Always run an unsupervised baseline alongside the LLM.** The KMeans baseline does not exist because we trust it more; it exists because it isolates what the LLM contributes. Where the two agree (banking, Wheat Board, federal-provincial), we are confident the signal is in the corpus and not in the model. Where they disagree (WWII), the disagreement is informative: the LLM is reading substance the vocabulary fails to encode. Treating either as a substitute for the other — using KMeans cluster sizes as a topical-attention measurement, or skipping the baseline and trusting the LLM blind — is the failure mode worth naming.